Глубокое обучение с подкреплением

Бонусный модуль 5: имитационное обучение в Godot

Курс Hugging Face Deep RL

Полная русскоязычная учебная версия на основе MDX

Источник: курс Hugging Face Deep RL (перевод полного английского текста).

Содержание

1 Введение **2**
- 1.1 Цели . . . 2
- 1.2 Предварительные знания и требования . . . 2

2 Среда **3**

3 Начало работы **3**
- 3.1 Откройте начальный проект в Godot . . . 3
- 3.2 Откройте сцену робота . . . 3
- 3.3 Нажмите значок скрипта рядом с AIController3D, чтобы открыть его для редактирования 4
- 3.4 Замените методы `get_obs()` и `get_reward()` приведённой ниже реализацией . . . 4
- 3.5 Замените методы `_physics_process()` и `reset()` приведённой ниже реализацией . . . 5
- 3.6 Замените методы `get_action_space()`, `get_action()` и `set_action()` приведённой ниже реализацией . . . 6
- 3.7 Замените метод `_input` приведённой ниже реализацией . . . 7
- 3.8 Откройте сцену записи демонстраций и выберите узел AIController3D . . . 7
- 3.9 Запишем несколько демонстраций . . . 8
- 3.10 Экспортируем игру для обучения . . . 9

4 Обучение нашего робота **9**
- 4.1 Скачайте скрипт имитационного обучения из репозитория Godot RL . . . 9
- 4.2 Запустите обучение со следующими аргументами . . . 9
- 4.3 Проверим обученного агента . . . 10
- 4.4 Откройте сцену ONNX-инференса . . . 10

5 (Необязательно) Как изменить среду **11**

6 Заключение **13**

1 Введение



Рис. 1: Обучение по демонстрациям в Godot RL Agents: обзор модуля [оригинал изображения]

Добро пожаловать в бонусный модуль, в котором вы **обучите агента-робота проходить уровень мини-игры с помощью имитационного обучения.**

К концу модуля **у вас будет обученный агент, способный проходить уровень так, как показано на видео**:

Видео: обученный робот проходит уровень с помощью инференса ONNX
Посмотреть исходное видео

1.1 Цели

- Научиться применять имитационное обучение с Godot RL Agents: обучить агента проходить мини-игру на основе записанных человеком экспертных демонстраций.

1.2 Предварительные знания и требования

- Перед началом рекомендуется пройти предыдущую главу (Godot RL Agents).
- Желательно некоторое знакомство с Godot, хотя для прохождения руководства писать код на GDScript не потребуется.
- Godot с поддержкой .NET (работоспособность проверена с 4.3.dev5 .NET; более новые версии тоже могут подойти).
- Godot RL Agents (в виртуальном окружении venv/conda можно выполнить `pip install godot-rl`).
- Библиотека imitation.
- Время: приблизительно 1–2 часа на выполнение проекта и обучение. В зависимости от оборудования это может занять больше или меньше времени.

2 Среда

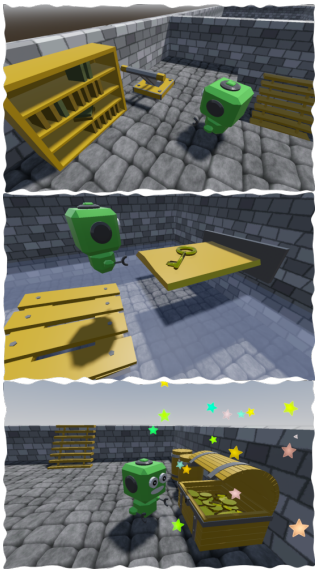


Рис. 2: Уровень мини-игры: рычаг, ключ и сундук с сокровищами [оригинал изображения]

В этой учебной среде робот должен выполнить следующие действия:

- Потянуть за рычаг, чтобы поднять лестницу, ведущую во вторую комнату.
- Добраться до ключа `[key]` и подобрать его, не упав в ловушки, воду или за пределы карты.
- Вернуться к сундуку с сокровищами в первой комнате и открыть его. Победа! `[trophy]`

3 Начало работы

Для начала скачайте проект по ссылке (нажмите значок загрузки рядом с файлом `GDRL-IL-Project.zip`). ZIP-архив содержит две версии проекта: «Starter» (начальная) и «Complete» (завершённая).

Игровая логика в начальном проекте уже реализована, а узлы настроены. Мы сосредоточимся на следующих задачах:

- Реализация кода узла AIController.
- Запись экспертных демонстраций.
- Обучение агента и экспорт файла .onnx для последующего инференса в Godot.

3.1 Откройте начальный проект в Godot

Распакуйте ZIP-архив, откройте Godot, нажмите «Import» и укажите папку `Starter\Godot` из распакованного архива.

3.2 Откройте сцену робота

> **Совет.** Сцену можно найти по слову «robot» через поиск FileSystem.

В этой сцене есть несколько разных узлов. Узел `robot` содержит визуальную модель робота. Узел `CameraXRotation` отвечает за поворот камеры мышью вверх и вниз при ручном управлении. ИИ-агент этим узлом не управляет: для обучения решению задачи он не нужен. Узел `RaycastSensors` содержит два датчика Raycast, которые позволяют агенту «ощущать» части игрового мира: стены, пол и т. п.

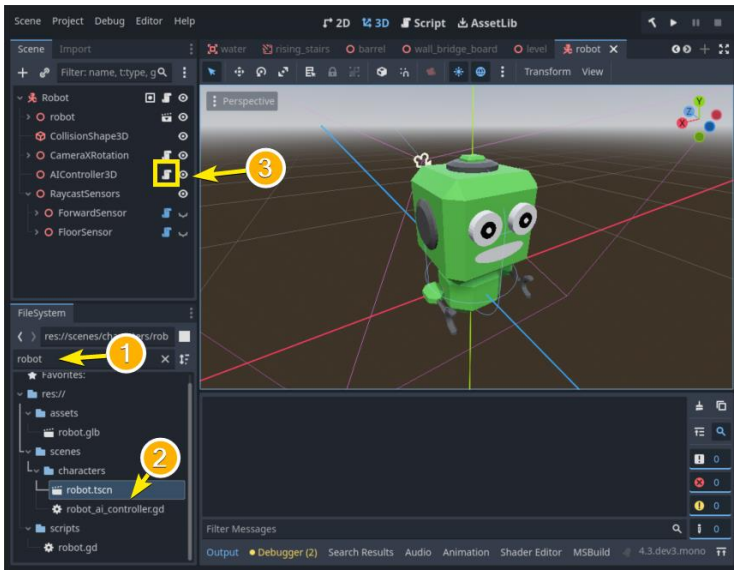


Рис. 3: Редактор Godot: открытие сцены робота [оригинал изображения]

3.3 Нажмите значок скрипта рядом с AIController3D, чтобы открыть его для редактирования

Совет. Чтобы проще найти узел, может потребоваться свернуть ветку «robot». Можно также ввести `aicontroller` в поле Filter над узлом `Robot`.

3.4 Замените методы `get_obs()` и `get_reward()` приведённой ниже реализацией

```
func get_obs() -> Dictionary:
    var observations: Array[float] = []
    for raycast_sensor in raycast_sensors:
        observations.append_array(raycast_sensor.get_observation())

    var level_size = 16.0

    var chest_local = to_local(chest.global_position)
    var chest_direction = chest_local.normalized()
    var chest_distance = clampf(chest_local.length(), 0.0, level_size)

    var lever_local = to_local(lever.global_position)
    var lever_direction = lever_local.normalized()
    var lever_distance = clampf(lever_local.length(), 0.0, level_size)

    var key_local = to_local(key.global_position)
    var key_direction = key_local.normalized()
    var key_distance = clampf(key_local.length(), 0.0, level_size)

    var raft_local = to_local(raft.global_position)
    var raft_direction = raft_local.normalized()
    var raft_distance = clampf(raft_local.length(), 0.0, level_size)

    var player_speed = player.global_basis.inverse() * player.velocity.limit_length(5.0) / 5.0

    (
        observations
        .append_array(
            [
                chest_direction.x,
                chest_direction.y,
                chest_direction.z,
                chest_distance,
```

```
                lever_direction.x,
                lever_direction.y,
                lever_direction.z,
                lever_distance,
                key_direction.x,
                key_direction.y,
                key_direction.z,
                key_distance,
                raft_direction.x,
                raft_direction.y,
                raft_direction.z,
                raft_distance,
                raft.movement_direction_multiplier,
                float(player._is_lever_pulled),
                float(player._is_chest_opened),
                float(player._is_key_collected),
                float(player.is_on_floor()),
                player_speed.x,
                player_speed.y,
                player_speed.z,
            ]
        )
    )
    return {"obs": observations}

func get_reward() -> float:
    return reward
```

В `get_obs()` мы сначала получаем наблюдения от двух датчиков Raycast, добавленных к узлу `AIController3D` в инспекторе, и включаем их в массив наблюдений. Затем вычисляем векторы относительного положения сундука, рычага, ключа и плота, разделяем их на направления и расстояния и также добавляем в наблюдения.

Кроме того, в наблюдения включается другая информация о состоянии игры:

- был ли потянут рычаг;
- подобран ли ключ;
- открыт ли сундук;
- стоит ли персонаж на полу (это также определяет, может ли он прыгнуть);
- нормализованная локальная скорость персонажа.

Булевы значения, например `_is_lever_pulled`, преобразуются в числа с плавающей точкой (0 или 1).

В методе `get_reward()` достаточно вернуть текущее вознаграждение.

3.5 Замените методы `_physics_process()` и `reset()` приведённой ниже реализацией

```
func _physics_process(delta: float) -> void:
    # Reset on timeout, this is implemented in parent class to set needs_reset to true,
    # we are re-implementing here to call player.game_over() that handles the game reset.
    n_steps += 1
    if n_steps > reset_after:
        player.game_over()

    # In training or onnx inference modes, this method will be called by sync node with actions
    provided,
    # For expert demo recording mode, it will be called without any actions (as we set the actions
    based on human input),
    # For human control mode the method will not be called, so we call it here without any actions
    provided.
```

```
    if control_mode == ControlModes.HUMAN:
        set_action()

    # Reset the game faster if the lever is not pulled.
    steps_without_lever_pulled += 1
    if steps_without_lever_pulled > 200 and (not player._is_lever_pulled):
        player.game_over()

func reset():
    super.reset()
    steps_without_lever_pulled = 0
```

3.6 Замените методы get_action_space(), get_action() и set_action() приведённой ниже реализацией

```
# Defines the actions for the AI agent ("size": 2 means 2 floats for this action)
func get_action_space() -> Dictionary:
    return {
        "movement": {"size": 2, "action_type": "continuous"},
        "rotation": {"size": 1, "action_type": "continuous"},
        "jump": {"size": 1, "action_type": "continuous"},
        "use_action": {"size": 1, "action_type": "continuous"}
    }

# We return the action values in the same order as defined in get_action_space() (important), but
    all in one array
# For actions of size 1, we return 1 float in the array, for size 2, 2 floats in the array, etc.
# set_action is called just before get_action by the sync node, so we can read the newly set values
func get_action():
    return [
        # "movement" action values
        player.requested_movement.x,
        player.requested_movement.y,
        # "rotation" action value
        player.requested_rotation.x,
        # "jump" action value (-1 if not requested, 1 if requested)
        -1.0 + 2.0 * float(player.jump_requested),
        # "use_action" action value (-1 if not requested, 1 if requested)
        -1.0 + 2.0 * float(player.use_action_requested)
    ]

# Here we set human control and AI control actions to the robot
func set_action(action = null) -> void:
    # If there's no action provided, it means that AI is not controlling the robot (human control),
    if not action:
        # Only rotate if the mouse has moved since the last set_action call
        if previous_mouse_movement == mouse_movement:
            mouse_movement = Vector2.ZERO

        player.requested_movement = Input.get_vector(
            "move_left", "move_right", "move_forward", "move_back"
        )
        player.requested_rotation = mouse_movement

        var use_action = Input.is_action_pressed("requested_action")
        var jump = Input.is_action_pressed("requested_jump")

        player.use_action_requested = use_action
        player.jump_requested = jump

        previous_mouse_movement = mouse_movement
    else:
        # If there is action provided, we set the actions received from the AI agent
```

```
        player.requested_movement = Vector2(action.movement[0], action.movement[1])
        # The agent only rotates the robot along the Y axis, no need to rotate the camera along X
    axis
        player.requested_rotation = Vector2(action.rotation[0], 0.0)
        player.jump_requested = bool(action.jump[0] > 0)
        player.use_action_requested = bool(action.use_action[0] > 0)
```

Метод `get_action()` (нужный только в режиме записи демонстраций) должен возвращать действия, которые агенту следует выполнять при встрече с тем же состоянием. Важно, чтобы значения лежали в допустимом диапазоне (`-1.0 to 1.0`), поэтому для булевых состояний используется выражение `-1 + 2 * variable`. Кроме того, действия должны идти в правильном порядке, заданном в `get_action_space()`.

В режиме записи демонстраций `set_action()` вызывается без аргумента действий: значения задаются по вводу человека. В режимах обучения и инференса этот метод получает аргумент `action`, содержащий значения всех действий от RL-модели. Поэтому оба случая обрабатываются конструкцией `if/else`.

Дополнительные пояснения приведены в комментариях к коду.

3.7 Замените метод `_input` приведённой ниже реализацией

```
# Record mouse movement for human and demo_record modes
# We don't directly rotate in input to allow for frame skipping (action_repeat setting) which
# will also be applied to the AI agent in training/inference modes.
func _input(event):
    if not (heuristic == "human" or heuristic == "demo_record"):
        return

    if event is InputEventMouseMotion:
        var movement_scale: float = 0.005
        mouse_movement.y = clampf(event.relative.y * movement_scale, -1.0, 1.0)
        mouse_movement.x = clampf(event.relative.x * movement_scale, -1.0, 1.0)
```

Этот фрагмент кода записывает движение мыши в режимах ручного управления и записи демонстраций.

Наконец, сохраните скрипт. Теперь можно переходить к следующему шагу.

3.8 Откройте сцену записи демонстраций и выберите узел AIController3D

> **Совет.** В поиске FileSystem можно ввести «demo», а в поле фильтрации узлов сцены — «aicontroller».

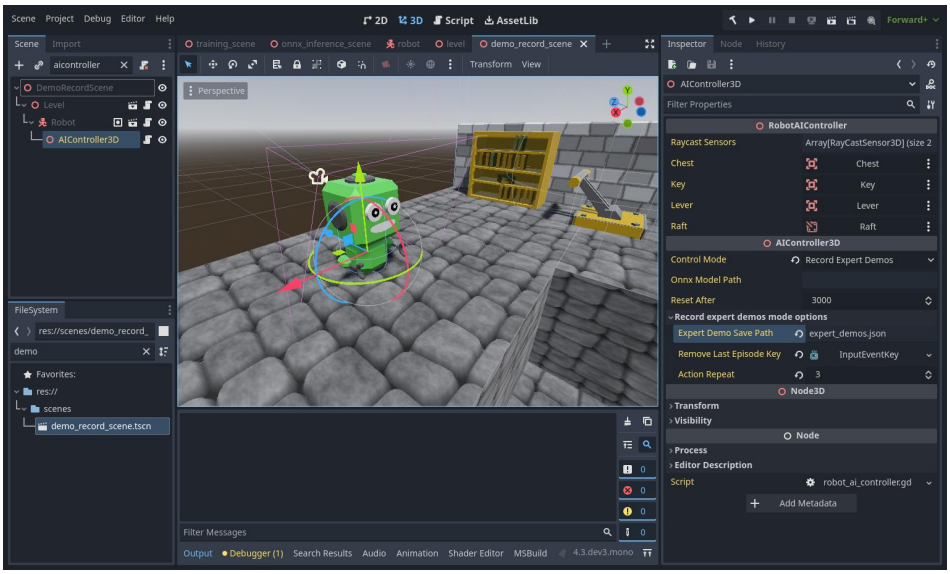


Рис. 4: Редактор Godot: сцена записи экспертных демонстраций [оригинал изображения]

Ничего менять не нужно: все параметры уже заданы. Но разберём настройки, которые потребовались бы для вашей собственной среды.

В сцене изменены настройки узла `Level > Robot > AIController3D`:

- Параметр `Control Mode` установлен в `Record Expert Demos`.
- Заполнено поле `Expert Demo Save Path`.
- Значение `Action Repeat` совпадает со значением узла `Sync` в сценах `training_scene` и `onnx_inference_scene`. Это значит, что каждое действие агента повторяется на протяжении трёх физических кадров. Настройка в `AIController` обеспечивает такое же повторение действий при управлении человеком (из-за чего возникает небольшая задержка), чтобы поведение соответствовало поведению агента. Значение довольно мало и не создаёт заметной задержки. Если решите его изменить, поменяйте его во всех трёх местах.
- Настройка `Remove Last Episode` позволяет назначить клавишу для удаления неудачного эпизода прямо во время записи, без перезапуска сеанса. Например, если робот упал в воду и игра сбросилась, этой клавишей можно удалить предыдущий эпизод, уже записывая следующий. По умолчанию выбрана `R`, но назначение можно изменить: нажмите на него, затем на кнопку `Configure`.

Ещё один способ упростить запись эпизодов в сложной среде — замедлить её во время записи. Для этого выберите узел `Sync` в сцене и измените свойство `Speed Up` (по умолчанию равно 1).

3.9 Запишем несколько демонстраций

Совет. Демонстрации сохраняются, только если записан хотя бы один полный эпизод, а окно игры закрыто кнопкой «X» или сочетанием ALT+F4. Кнопка остановки в редакторе Godot не сохраняет демонстрации. Лучше сначала записать один эпизод и проверить, появился ли файл `expert_demos.json` в файловой системе или папке проекта Godot.

Убедитесь, что открыта сцена `demo_record_scene`, и `press F6`, чтобы начать запись демонстраций.

Управление:

- мышь управляет камерой (если нужно изменить чувствительность, откройте сцену `robot`, выберите узел `Robot` и измените параметр `Rotation Speed`; сохраняйте одинаковое значение при записи, обучении и инференсе);
- клавиши `WASD` отвечают за перемещение персонажа;

- `SPACE` — прыжок;
- `E` — активация рычага и открытие сундука.

Сначала можете немного потренироваться, чтобы привыкнуть к среде. Если хотите пропустить запись демонстраций, в завершённом проекте уже есть готовый файл `expert_demos.json`, которым можно воспользоваться.

Записанные демонстрации должны содержать не менее 22–24 полных успешных эпизодов. При обучении можно использовать несколько файлов демонстраций, поэтому необязательно записывать всё за один раз (имя файла задаётся упомянутым выше параметром `Expert Demo Save Path`).

Запись 23 эпизодов заняла у автора около ~10 минут. Ключ появляется поочерёдно в двух местах, поэтому 22 или 24 эпизода обеспечили бы равномерное распределение его положений в демонстрациях, хотя и 23 достаточно близко. Подходя к рычагу или сундуку, автор удерживал клавишу `E` чуть дольше, чтобы действие попало в несколько шагов рядом с объектом. Несколько неудачных эпизодов он удалил клавишей `R` во время записи следующего эпизода.

Ниже — ускоренное видео процесса записи демонстраций:

Видео: запись экспертных демонстраций человеком (ускоренная съёмка)
Посмотреть исходное видео

3.10 Экспортируем игру для обучения

Экспортировать игру из Godot можно через меню `Project > Export`.

4 Обучение нашего робота

Совет. Прежде чем начать обучение, установите библиотеку imitation в то же виртуальное окружение venv или conda, в котором установлен Godot RL Agents. Для этого выполните `pip install imitation`.

4.1 Скачайте скрипт имитационного обучения из репозитория Godot RL

4.2 Запустите обучение со следующими аргументами

```
sb3_imitation.py --env_path="path_to_ILTutorial_executable" --bc_epochs=100 --gail_timesteps
    =1450000 --demo_files "path_to_expert_demos.json" --n_parallel=4 --speedup=20 --
    onnx_export_path=model.onnx --experiment_name=ILTutorial
```

Укажите в параметре пути к среде путь до экспортированной игры, а в параметре файлов демонстраций — путь к записанным данным. Если файлов несколько, перечислите их через пробел, например `--demo_files demos.json demos2.json`**.**

Можно также задать большое число шагов в `--gail_timesteps`, а затем вручную остановить обучение сочетанием `CTRL+C`. Автор остановил обучение таким способом, когда вознаграждение стало приближаться к 3, на отметке `total_timesteps | 1.38e+06`. Это заняло примерно ~41 минуту, а предварительное обучение BC — около ~5,5 минуты на CPU его компьютера.

Чтобы видеть среду во время обучения, добавьте аргумент `--viz`. На этапе BC среда будет неподвижна: для этого этапа она не используется, кроме получения информации о пространствах наблюдений и действий. На этапе GAIL визуализация среды начнёт обновляться.

Ниже приведены показатели `ep_rew_mean` и `ep_rew_wrapped_mean` из логов, отображённые в TensorBoard. В данном случае их графики довольно близки:

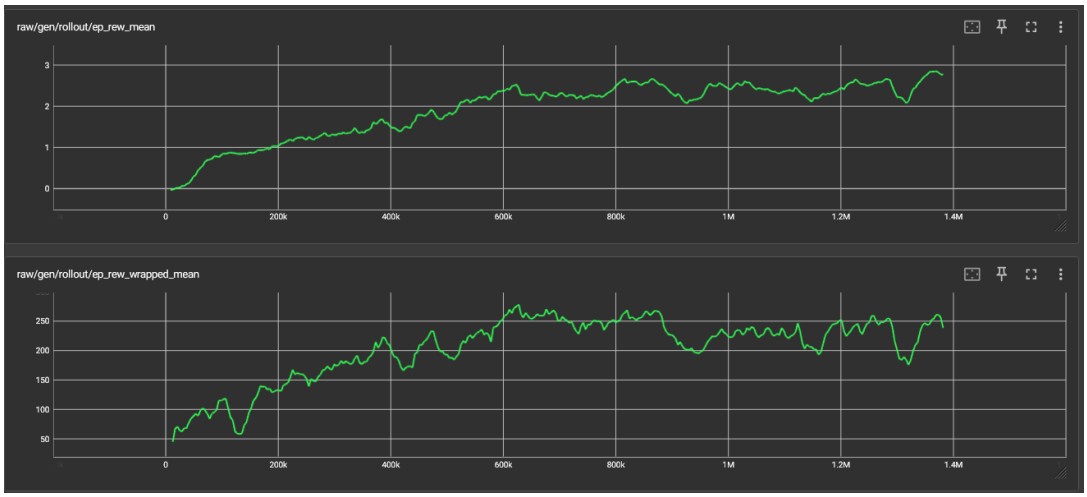


Рис. 5: Кривые обучения: ep_rew_mean и ep_rew_wrapped_mean в TensorBoard [оригинал изображения]

Совет. Логи находятся в папке `logs/ILTutorial` относительно каталога, из которого запускалось обучение. Если выполняете несколько запусков, каждый раз меняйте аргумент `--experiment_name`.

Хотя для самого обучения здесь не нужно задавать вознаграждения среды (и они не используются), для контроля успешности реализована простая разреженная функция вознаграждения. Падение за пределы карты, в воду или ловушки задаёт `reward += -1`, а активация рычага, подбор ключа и открытие сундука дают по `reward += 1`. Если `ep_rew_mean` приближается к 3, результат хороший. Показатель `ep_rew_wrapped_mean` — вознаграждение от дискриминатора GAIL, которое напрямую не показывает, насколько успешно агент решает задачу.

4.3 Проверим обученного агента

После обучения в папке, из которой запускался скрипт, появится файл `model.onnx` (полный путь к файлу `.onnx` также можно найти ближе к концу лога обучения в консоли). **Скопируйте его в папку игрового проекта Godot.**

4.4 Откройте сцену ONNX-инференса

Как и сцена записи демонстраций, эта сцена использует только один экземпляр уровня. Для её узла `Sync` задан режим `Onnx Inference`.

Выберите узел `Sync` и задайте для свойства `Onnx Model Path` значение `model.onnx`.

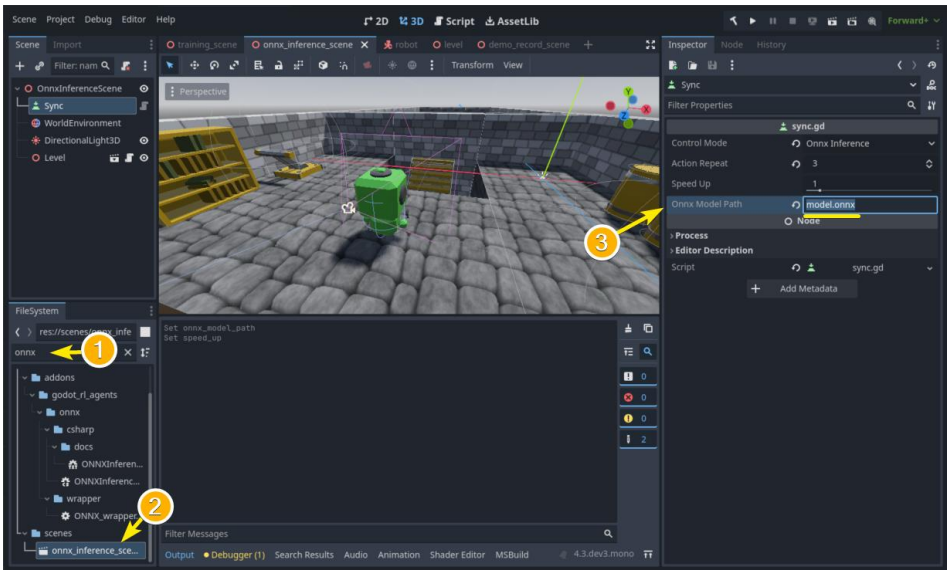


Рис. 6: Редактор Godot: настройка сцены ONNX-инференса [оригинал изображения]

Нажмите F6, чтобы запустить сцену, и посмотрим, чему научился агент!

Видео работы обученного агента:

> **Видео:** обученный робот проходит уровень с помощью инференса ONNX
> Посмотреть исходное видео

Судя по результатам, агент умеет подбирать ключ в обоих возможных положениях (на левой или правой платформе) и хорошо воспроизводит записанное поведение. **Если получили похожий результат — отлично, вы успешно прошли это руководство!** `[trophy]` `[applause]`

Если результаты существенно отличаются, учтите, что на них влияют количество и качество демонстраций. Может помочь изменение числа шагов на этапах BC/GAIL и гиперпараметров Python-скрипта. Есть и вариации от запуска к запуску, поэтому даже при одинаковых настройках результаты иногда немного различаются.

5 (Необязательно) Как изменить среду

Если хотите изменить игровой уровень, откройте сцену `res://scenes/level.tscn`, затем в FileSystem Godot откройте папку `res://scenes/modules/`:

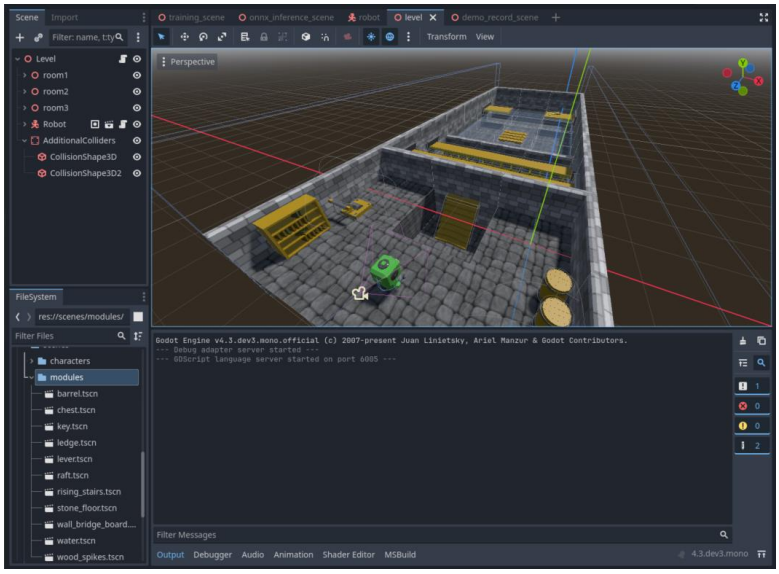


Рис. 7: Сцена уровня и переиспользуемые модули комнат в Godot [оригинал изображения]

Уровень содержит три комнаты, собранные из модулей, робота и дополнительные коллайдеры. Последние не позволяют пройти уровень обходным способом: забраться на стену первой комнаты и оттуда добраться до ключа. Добавляя модули в сцену, можно создавать новые комнаты и предметы.

Если выбрать узел Key (он находится в `Room3`, но его можно найти и поиском), а затем открыть `Node > Signals`, станет видно, что сигнал `collected` подключён и к роботу, и к сундуку. Это позволяет отслеживать подбор ключа и разблокировать сундук. Такой же механизм используется для активации лестницы рычагом. Если добавить новые рычаги, лестницы или ключи, их можно соединить сигналами.

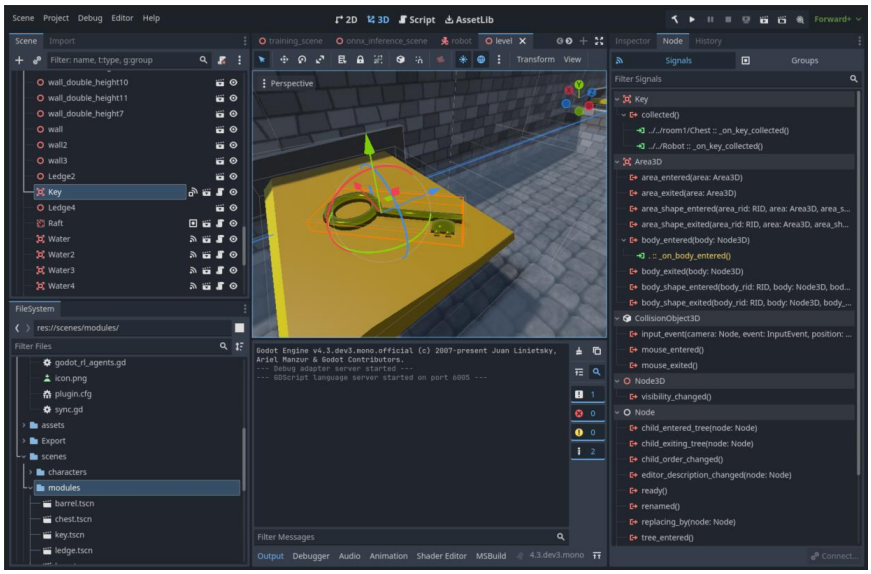


Рис. 8: Сигналы Godot: связывание собираемых предметов и событий уровня [оригинал изображения]

Если переключиться на `Groups`, видно, что ключ входит в группу `resetable`. В эту же группу входят плот, рычаг, сундук и персонаж. Сюда можно добавить любой узел, который требуется сбрасывать при перезапуске эпизода.

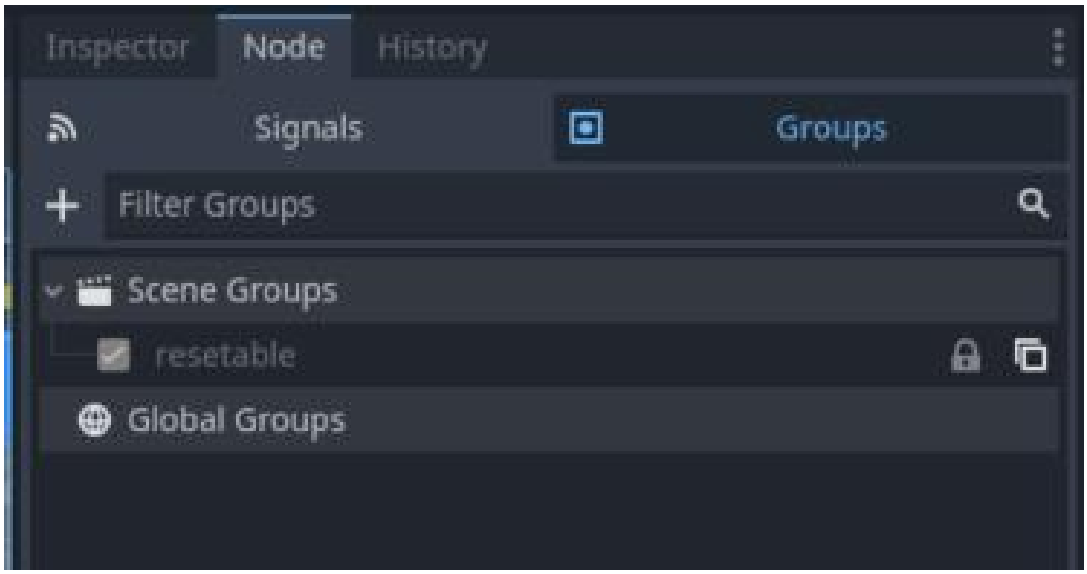


Рис. 9: Узлы группы resetable, сбрасываемые при перезапуске эпизода [оригинал изображения]

Для корректной работы каждый объект группы `resetable` должен реализовать метод `reset()`, отвечающий за его сброс.

Поскольку во время обучения используется несколько экземпляров сцены уровня, мы сбрасываем не все объекты `resetables`, а только находящиеся внутри соответствующего экземпляра сцены.

За это отвечает метод `reset_all_resetables()` в файле `level_manager.gd`. Скрипт робота вызывает его при необходимости сброса.

После изменения размера уровня необходимо обновить переменную `level_size` в файле `robot_ai_controller.gd`. Приблизительно измерьте наибольший размер уровня и запишите это значение в переменную.

Если изменяется количество объектов, которые должен отслеживать `AIController` (рычагов, плотов и т. п.), нужно соответствующим образом изменить код скрипта, добавить экспортируемые свойства для этих объектов, а затем привязать их через свойства инспектора `AIController` в сцене уровня:

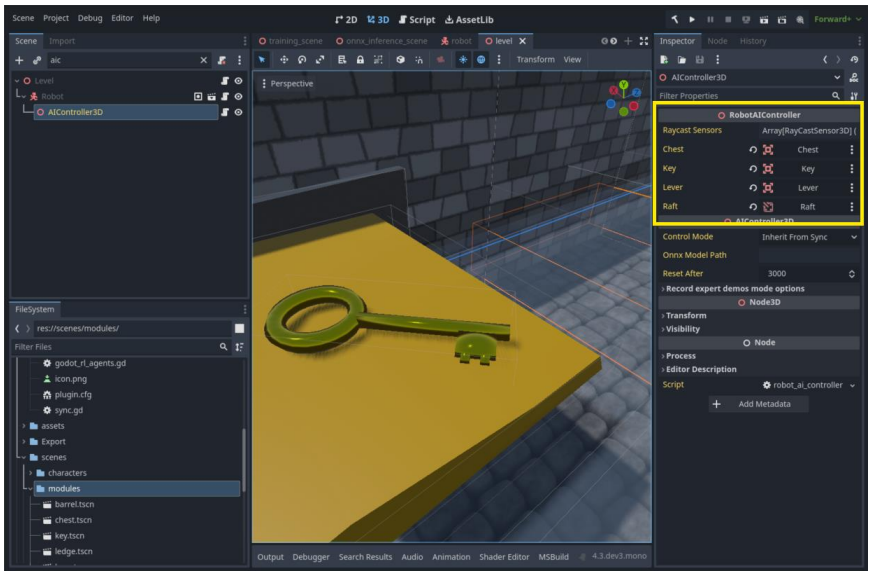


Рис. 10: Инспектор Godot: ссылки AIController на объекты уровня [оригинал изображения]

После этого, возможно, придётся обновить такие же свойства `AIController` в сцене записи демонстраций.

6 Заключение

Поздравляем с завершением этого бонусного модуля! Вы познакомились с процессом записи экспертных демонстраций и обучения агента методом имитационного обучения (IL). В некоторых задачах это может быть альтернативой обучению игровых агентов методами RL.

Автор руководства — Иван Додич (Ivan Dodic). Благодарим Эдварда Бичинга (Edward Beeching) и Томаса Симонини (Thomas Simonini) за рецензирование и отзывы.